CS228: Introduction to Deep Learning

Lecture 9: initialization and optimizers

Instructor: Zhouxing Shi

Project proposal (**Due April 19**)

- Decide a task you will work on for the project
- Submit a brief writeup (0.5 - 1 page)
- Cover:
  - Introduction and background of the task
  - A high-level overview of existing methods
  - Datasets and metrics to use
- No new method needed at this stage
- Grading: if the proposal is reasonably completed.

Midterm (**April 23-24**)

- Scope: Week 1 - 3
- You should understand:
  - Core algorithms (perceptron learning, gradient descent)
  - MLP (with linear layers and ReLU), Softmax, Cross Entropy
  - Basic PyTorch coding in Assignment 1
- You *do not* have to memorize complicated formulations or results that cannot be quickly derived, such as:
  - Convergence result/proof of perceptron learning
  - Formulation of advanced optimizers in this class

Midterm (**April 23~24**)

- Location: CSE test center at **WCH133**
- Reserve a 50-min window on PrairieTest **(now open)**
  - See https://testcenter.cs.ucr.edu/ for more information
- Paper-based. Closed-book.
- Question types: short answers/derivations, code review
- No lecture on April 24

Initialization

- In perceptron learning, we initialized weights to 0
- Would it work for MLP with ReLU?

Whiteboard: zero init

Initialization

- Random weight initialization is needed for NN
  - Use normal distribution or uniform distribution
- Principle:
  - Keep the variance stable across layers
  - The variance of initial weight matrices depends on the shape of the layer

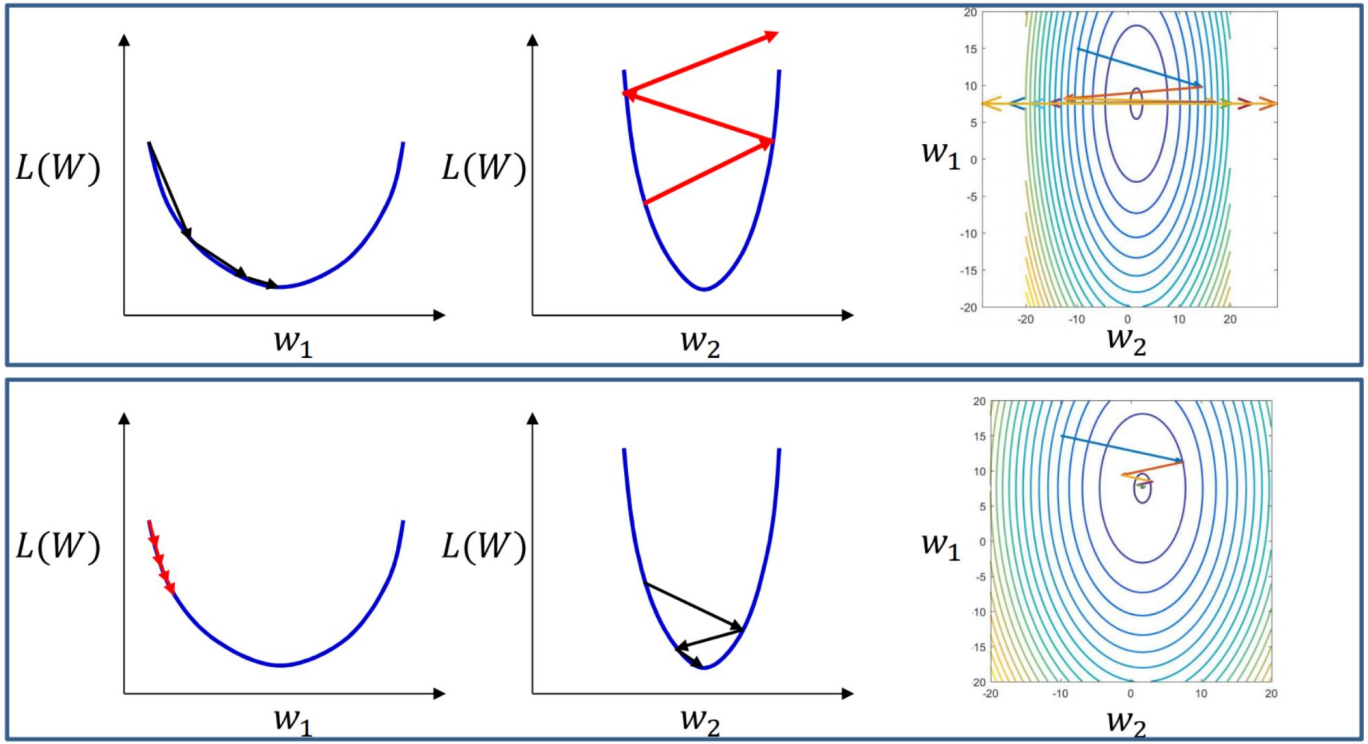
$$\sigma_{\text{Xavier}} = \sqrt{\frac{2}{n_{\text{in}} + n_{\text{out}}}} \qquad \sigma_{\text{Kaiming}} = \sqrt{\frac{2}{n_{\text{in}}}}$$

Limitation of plain gradient descent

- A large number of parameters are optimized
- Plain gradient descent applies **a single learning rate to all the**

$$\mathbf{W}^{(t)} \leftarrow \mathbf{W}^{(t-1)} - \alpha \nabla_{\mathbf{W}^{(t-1)}} f(\mathbf{W}^{(t-1)})$$

Limitation of plain gradient descent



It converges in some directions but oscillate or diverge in others

The fix

- Use different “effective” learning rates for different directions (parameters)
  - Use larger learning rates for directions that converge smoothly
  - Use smaller learning rates for directions that bounce around

Momentum

- Track oscillations: maintain an average of gradients
  - Large for directions with smooth convergence
  - Small for directions with swings (positive and negative gradients cancelling out)

Momentum

- Update parameters using the *exponential moving average* rather than the current gradients

Plain GD $$\mathbf{W}^{(t)} \leftarrow \mathbf{W}^{(t-1)} - \alpha \nabla_{\mathbf{W}^{(t-1)}} f(\mathbf{W}^{(t-1)})$$

GD with momentum e.g., β=0.9

$$\Delta \mathbf{W}^{(t)} = \beta \Delta \mathbf{W}^{(t-1)} - \alpha \nabla_{\mathbf{W}^{(t-1)}} f(\mathbf{W}^{(t-1)})$$

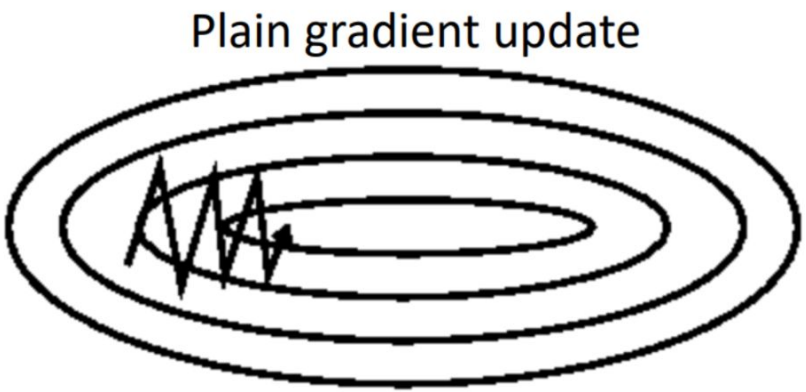
$$\mathbf{W}^{(t)} \leftarrow \mathbf{W}^{(t-1)} + \Delta \mathbf{W}^{(t)}$$

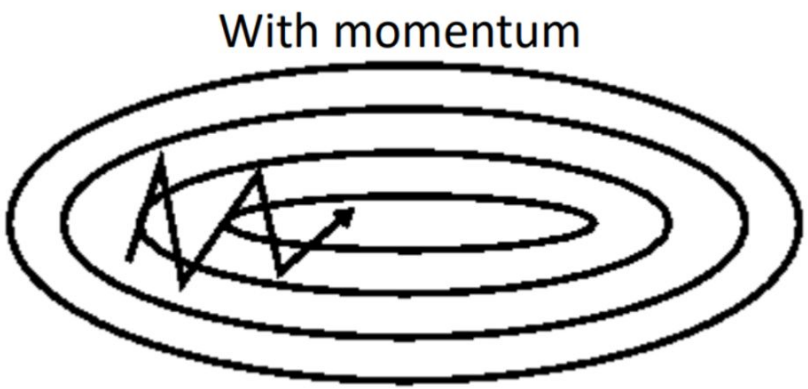
Exponential moving average (geometrically weighted sum):

$$\Delta \mathbf{W}^{(t)} = -\alpha \sum_{i=1}^{t-1} \beta^{t-1-i} \nabla_{\mathbf{W}^{(i)}} f(\mathbf{W}^{(i)})$$

Momentum

- Larger step size where gradients stay in the same sign
- Smaller step size where the sign keeps flipping





Nestorov's Accelerated Gradient

- Change the order:
  - First extend the previous step
  - Then calculate the gradient
  - Update the parameters with the above two combined

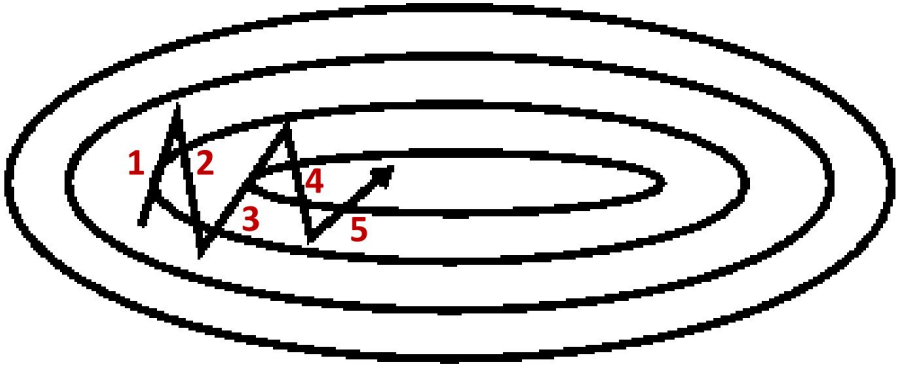
$$\Delta \mathbf{W}^{(t)} = \beta \Delta \mathbf{W}^{(t-1)} - \alpha \nabla_{\mathbf{W}^{(t-1)}} f(\mathbf{W}^{(t-1)} + \beta \Delta \mathbf{W}^{(t-1)})$$

looking ahead at the future gradient

$$\mathbf{W}^{(t)} \leftarrow \mathbf{W}^{(t-1)} + \Delta \mathbf{W}^{(t)}$$

Adaptive learning rates

- A learning rate for each parameter
- Directions with larger oscillation:
  - Have larger movements
  - Need smaller learning rates
- Adjust learning rates based on cumulative movements



RMSProp

- Accumulate squared gradients by exponential moving average:

$$r_i^{(t)} \leftarrow \beta r_i^{(t-1)} + (1-\beta)\left(g_i^{(t-1)}\right)^2$$

- Update each parameter:

$$w_i^{(t)} \leftarrow w_i^{(t-1)} - \frac{\alpha}{\sqrt{r_i^{(t)}} + \epsilon} g_i^{(t-1)}$$

Adaptive learning rate is inversely proportional to the cumulative squared gradient

Adam

- Adam = RMSProp + Momentum

First momentum (follows GD with momentum)

$$m_i^{(t)} = \beta_1 m_i^{(t-1)} + (1-\beta_1) g_i^{(t-1)}$$

Second momentum (follows RMSProp)

$$v_i^{(t)} = \beta_2 v_i^{(t-1)} + (1-\beta_2) \left(g_i^{(t-1)}\right)^2$$

Adam

- Adam = RMSProp + Momentum

$$\hat{m}_i^{(t)} = \frac{m_i^{(t)}}{1-\beta_1^t} \qquad \hat{v}_i^{(t)} = \frac{v_i^{(t)}}{1-\beta_2^t}$$ (Bias correction terms)

$$w_i^{(t)} \leftarrow w_i^{(t-1)} - \frac{\alpha}{\sqrt{\hat{v}_i^{(t)}} + \epsilon} \hat{m}_i^{(t)}$$